

# Lab 04: Feature Extraction

Turn an image into a small set of numbers a classifier can use. This guide covers the six histogram methods, filtering and convolution, and edge detection with Canny, with every code block in the manual explained line by line.

ure

HOG

LBP

Color hist

HED

HIG

Energy & contrast

Filtering

Edges

Canny

Lab 1

◀

▶

## The big picture

Manual pp. 3-5

**Feature extraction** converts raw pixels (huge, noisy, high-dimensional) into a compact vector that keeps what matters and drops the rest. Everything in this lab produces such a vector, most often a histogram.

Local features

From small regions: corners, keypoints, patches. Used for matching and object detection.

Global features

From the whole image: color histograms, texture descriptors, moments. One vector per image.

**Technique families:** histogram-based (this lab's section 3), filter-based (Gabor, Haar, convolution kernels), and learned features from CNNs.

**Used for:** object, face and gesture recognition, medical imaging, autonomous vehicles. **Hard because of:** changes in lighting, scale, orientation and noise.

### ONE-GLANCE COMPARISON

| METHOD          | MEASURES                | BUILT FROM                                          | OUTPUT VECTOR                       | BEST FOR                        |
|-----------------|-------------------------|-----------------------------------------------------|-------------------------------------|---------------------------------|
| HOG             | Edge direction per cell | Sobel gradients, 8x8 cells, 2x2 block normalization | Long vector (size depends on image) | Pedestrian and object detection |
| LBP             | Local texture pattern   | Neighbor vs center comparison                       | 10 values (8 points, uniform)       | Texture classification, faces   |
| Color histogram | Color distribution      | Bins in RGB or HSV                                  | Bin counts                          | Color-based retrieval           |
| HED             | Dominant edge           | Gaussian, Sobel angle                               | 8 values (45° bins)                 | Texture,                        |

|                   | directions                  |                       |                        | segmentation            |
|-------------------|-----------------------------|-----------------------|------------------------|-------------------------|
| HIG               | Gradient orientation spread | Sobel angle           | 9 values (20° bins)    | Texture, classification |
| Energy / contrast | Smoothness / variation      | 3x3 neighborhood sums | Two 256-bin histograms | Texture analysis        |

**Memory hook.** HOG, HED and HIG all start from the same two Sobel images (dx, dy) and an angle. They differ only in how they bin and whether they work per cell or per image.

### 3.1 HOG: Histogram of Oriented Gradients

Manual pp. 5-6

**Idea:** an object's shape is described well by the directions of its edges. HOG splits the image into small cells, builds a histogram of edge directions inside each cell, normalizes groups of cells, and joins everything into one vector.

**STEPS**

- 1. Convert to grayscale.
- 2. Compute the gradient magnitude and direction per pixel with Sobel.
- 3. Divide into non-overlapping cells (8x8 pixels).
- 4. Per cell, build an orientation histogram (bins such as 0-20°, 20-40°). Each pixel votes by its gradient.
- 5. Group cells into blocks (2x2 cells) and normalize each block's histograms. This cancels lighting changes.
- 6. Concatenate all normalized block histograms into the feature vector.

**CODE, LINE BY LINE**

```
from skimage.feature import hog # the HOG implementation
from skimage import exposure # used only to brighten the picture for display
import cv2, matplotlib.pyplot as plt

image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE) # step 1: grayscale 2D array

features, hog_image = hog(image,
    pixels_per_cell=(8, 8), # cell size (step 3)
    cells_per_block=(2, 2), # block size for normalization (step 5)
    visualize=True) # also return a picture of the gradients
# features = the final 1D vector (step 6). hog_image = per-cell edge picture.

hog_image_rescaled = exposure.rescale_intensity(hog_image, in_range=(0, 10))
# HOG pictures are dark; stretch values 0..10 to full range so you can see them
```

```
plt.subplot(121); plt.imshow(image, cmap=plt.cm.gray)           # original
plt.subplot(122); plt.imshow(hog_image_rescaled, cmap=plt.cm.gray) # HOG view
plt.show()
print(features)                                     # the feature vector
```

**Watch out.** Vector length depends on image size (cells x blocks x 9 orientations by default). Resize all images to one size before comparing or training. The manual's PDF cuts off the `cells_per_block` line, but it is `(2, 2)`.

## 3.2 LBP: Local Binary Pattern

Manual pp. 6-8

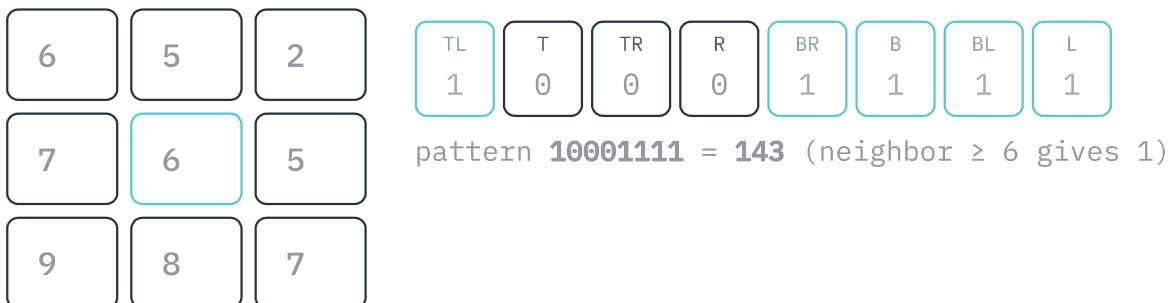
**Idea:** describe texture by comparing each pixel with its neighbors. Each neighbor becomes a 1 (neighbor  $\geq$  center) or a 0. The bits form a number, and the histogram of those numbers over the image is the texture signature.

### STEPS

1. Grayscale.
2. Pick a circular neighborhood: radius and number of points.
3. For each neighbor: 1 if it is  $\geq$  the center, else 0.
4. Join the bits into one binary pattern, which is a number.
5. Histogram those numbers over the whole image.
6. The histogram is the feature vector.

### TRY IT: ONE PIXEL'S LBP CODE

Edit the numbers. The center is highlighted. Neighbors are read clockwise from the top-left.



### CODE, LINE BY LINE

```
from skimage import feature
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)

radius = 1                # neighbors sit 1 pixel from the center
n_points = 8 * radius     # 8 neighbors for radius 1 (rule of thumb: 8 x radius)
```

```

lbp_image = feature.local_binary_pattern(image, n_points, radius, method='uniform')
# Replaces every pixel with its LBP code.
# 'uniform' keeps only patterns with at most 2 bit changes (0->1 or 1->0).
# All other patterns share one "non-uniform" label, so codes run 0..n_points+1 (0..9).

hist, _ = np.histogram(lbp_image.ravel(), bins=np.arange(0, n_points + 3), range=(0, n_points+1))
# ravel() flattens to 1D. Bin edges 0..10 give 10 bins, one per possible code.

hist = hist.astype("float")
hist /= (hist.sum() + 1e-6) # normalize to proportions; 1e-6 avoids divide-by-zero.
                           # Now images of different sizes are comparable.

plt.bar(range(0, n_points + 2), hist) # bar chart of the 10-value feature vector

```

**Why uniform?** The 256 raw patterns collapse into 10 meaningful bins (edges, corners, flat spots, spots). A shorter vector means less noise and faster classifiers.

## 3.3 Histogram of Color

Manual p. 8 (no code given)

**Idea:** count how many pixels fall in each color range. Split the color space (RGB or HSV) into bins and count pixels per bin. It is a global feature that ignores where the colors are.

```

img = cv2.imread('image.jpg') # BGR color image
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV) # HSV is less sensitive to brightness
hist = cv2.calcHist([hsv], [0,1,2], None, [8,8,8], [0,180, 0,256, 0,256])
hist = cv2.normalize(hist, hist).flatten() # 8x8x8 = 512-value vector

```

This snippet is an addition. The manual only describes the idea.

## 3.4 HED: Histogram of Edge Directions

Manual pp. 8-10

**Idea:** count how many pixels have edges pointing in each of 8 directions (45° apart). It tells you the dominant edge directions: horizontal-heavy, vertical-heavy, diagonal, and so on.

### STEPS

1. Grayscale, optionally Gaussian blur to cut noise.
2. Edge detection (Canny, Sobel or Scharr) gives an edge map.
3. Gradient angle per pixel with `arctan2`.

4. Bin angles into 8 bins covering 360°

4. Bin angles into 8 bins covering 360°.

5. Count pixels per bin. The counts are the feature vector.

#### CODE, LINE BY LINE

```
image_smoothed = cv2.GaussianBlur(image, (5, 5), 0)      # 5x5 Gaussian blur; sigma 0 = auto
edges = cv2.Canny(image_smoothed, threshold1=30, threshold2=70)
# low=30, high=70 for hysteresis. Used here only for the displayed edge map.

gradient_x = cv2.Sobel(image_smoothed, cv2.CV_64F, 1, 0, ksize=3)  # dI/dx (vertical edges)
gradient_y = cv2.Sobel(image_smoothed, cv2.CV_64F, 0, 1, ksize=3)  # dI/dy (horizontal edges)
# CV_64F = 64-bit float so negative slopes are not clipped to 0

gradient_magnitude = np.sqrt(gradient_x**2 + gradient_y**2)        # edge strength (not used)
gradient_orientation = np.arctan2(gradient_y, gradient_x) * 180 / np.pi  # angle in degrees

hist, bin_edges = np.histogram(gradient_orientation, bins=8, range=(0, 360))
# 8 bins of 45 degrees: 0-45, 45-90 ... 315-360

plt.bar(bin_edges[:-1], hist, width=45, align='center')  # bar chart of directions
plt.xticks(range(0, 360, 45))
```

**Bug in the manual.** `arctan2` returns angles from -180 to 180, but the histogram range is 0 to 360. Every pixel with a negative angle is silently dropped. In my test, only 222 of 400 pixels were counted. Fix:

```
gradient_orientation % 360.
```

## 3.5 HIG: Histogram of Intensity Gradients

Manual pp. 10-12

**Idea:** the same as HED but on the whole image with 9 bins over 0-180°, with optional normalization. Where HOG keeps spatial layout (per-cell histograms), HIG is one global histogram.

#### STEPS

1. Grayscale, optional resize, smooth or contrast stretch.
2. Gradients  $dx$ ,  $dy$  with Sobel or Scharr.
3. Magnitude  $G = \sqrt{dx^2 + dy^2}$ , orientation  $\theta = \text{atan2}(dy, dx)$ .
4. Bin orientations (fixed or adaptive number of bins).
5. Normalize the histogram so it is scale-invariant, and use it as the vector.

#### CODE, LINE BY LINE

```
dx = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3)      # horizontal gradient
dy = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3)      # vertical gradient
```

```

magnitude = np.sqrt(dx**2 + dy**2)           # how strong the change is
orientation = np.arctan2(dy, dx) * (180 / np.pi) # radians to degrees

histogram, bins = np.histogram(orientation, bins=9, range=(0, 180)) # 9 bins of 20 degrees
histogram = histogram / histogram.sum()      # proportions that add up to 1

plt.bar(bins[:-1], histogram, width=20)      # plot the 9-value feature vector

```

**Same bug as HED, and a weakness.** Angles run -180 to 180 but the range is 0-180, so negative angles are dropped. Fix: `orientation % 180` (unsigned gradients, as HOG does). Also, each pixel counts as 1 regardless of edge strength. Passing `weights=magnitude` to `np.histogram` makes strong edges count more.

## 3.6 Texture energy and contrast histograms

Manual pp. 12-14

### Texture energy

$E = \sum(\text{pixel values})^2$  in a window. Measures how much signal is in the neighborhood. In the manual's description, higher means a more complex, less uniform texture.

### Texture contrast

$C = \sqrt{\text{Var}(\text{pixel values})}$  in a window, which is the standard deviation. Higher means more variation, so a rougher texture.

### STEPS

1. Grayscale, optional resize, smooth or contrast stretch.
2. Choose a 3x3 or 5x5 window.
3. For every pixel, compute energy and contrast over its window.
4. Histogram each map into bins.
5. The two histograms are the feature vectors.

### CODE, LINE BY LINE

```

neighborhood_size = 3                                # 3x3 window

energy = cv2.filter2D(image ** 2, -1, np.ones((neighborhood_size, neighborhood_size)))
# image**2 squares every pixel; filter2D with all-ones kernel sums each 3x3 window.
# Result: sum of squares per window = texture energy.
contrast = cv2.filter2D(image, -1, np.ones((neighborhood_size, neighborhood_size)))
# all-ones kernel on the raw image = window SUM (used as the "contrast" map)

energy_histogram, energy_bins = np.histogram(energy, bins=256, range=(0, energy.max()))
contrast_histogram, contrast_bins = np.histogram(contrast, bins=256, range=(0, contrast.max()))

```

07/10/2026, 17:38Lab 04 Feature Extraction Guide

```
contrast_histogram, contrast_bins = np.histogram(contrast, bins=256, range=(0, contrast.max()),
# 256-bin histogram of each map, scaled to its own maximum

energy_histogram = energy_histogram / energy_histogram.sum() # normalize to proportion
contrast_histogram = contrast_histogram / contrast_histogram.sum()

plt.plot(energy_bins[:-1], energy_histogram, color='b') # blue curve
plt.plot(contrast_bins[:-1], contrast_histogram, color='r') # red curve
```

**Two bugs in the manual.** (1) `image ** 2` on a uint8 image wraps around (for example  $20^2$  becomes 144), and `filter2D` with `-1` clips the output to 0-255, so the energy map is wrong. Convert first: `f = image.astype(np.float32)`. (2) The "contrast" line computes a window sum, not the standard deviation the text defines. Correct version: `mean = cv2.blur(f,(3,3)); contrast = np.sqrt(np.maximum(cv2.blur(f*f,(3,3)) - mean**2, 0))`.

## 4. Filtering and convolution

Manual pp. 15-18

**Filter (kernel):** a small matrix of weights. **Convolution:** slide the kernel over the image. At each position, multiply the kernel by the pixels under it, add up the products (a dot product), and write the sum to the output pixel. Used for blur, noise reduction, edges, sharpening and feature extraction.

### KERNELS IN THE MANUAL

| KERNEL         | EFFECT                                                                          | CODE AND WHAT IT DOES                                                                                                                                    |
|----------------|---------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Box blur       | Each pixel becomes the average of its 3x3 neighborhood. Removes noise, smooths. | <code>np.ones((3,3), np.float32) / 9</code> . Nine equal weights that sum to 1, so brightness is unchanged.                                              |
| Gaussian blur  | Weighted average, center counts most. Smoother than box.                        | <code>cv2.getGaussianKernel(5, 1.0)</code> builds the weights. <code>sigma = 1.0</code> sets the spread.                                                 |
| Sobel / Scharr | Highlights rapid intensity change (edges). One kernel for x, one for y.         | <code>cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3):dx=1, dy=0</code> gives vertical edges. <code>cv2.Scharr(...)</code> is the more accurate 3x3 variant. |
| Emboss         | 3D relief look from neighbor differences.                                       | Kernel <code>[[ -2, -1, 0 ], [ -1, 1, 1 ], [ 0, 1, 2 ]]</code> . Negative weights on one side, positive on the other.                                    |

```
blurred_image = cv2.filter2D(image, -1, kernel)
# filter2D(source, depth, kernel). depth -1 = output has the same data type as the input.
# One call, any kernel: that is why every example above only changes 'kernel'.
```

**Gaussian bug.** `cv2.getGaussianKernel(5, 1.0)` returns a 5x1 column, not a 5x5 matrix, so `filter2D` with it blurs vertically only. Use `cv2.GaussianBlur(image, (5,5), 1.0)`, or build 2D with `k = k @ k.T`.

## 4.2.1 Convolution types

| TYPE                      | MEANING                                                                                                | OUTPUT SIZE                                 |
|---------------------------|--------------------------------------------------------------------------------------------------------|---------------------------------------------|
| <b>Standard (full)</b>    | Center the kernel on every pixel, multiply and sum.                                                    | Same as input in the manual's code          |
| <b>Valid</b>              | Only positions where the kernel fully overlaps the image. No padding.                                  | Smaller (3x3 kernel: lose 1 pixel per side) |
| <b>Same</b>               | Pad with zeros around the border so edges are processed too. Prevents information loss at the borders. | Same as input                               |
| <b>Valid with strides</b> | Valid, but jump 2 pixels at a time instead of 1.                                                       | Smaller still (about half for stride 2)     |

Manual code for all four uses the same 3x3 kernel `[[1,0,-1],[2,0,-2],[1,0,-1]]`, which is the Sobel x kernel. It finds vertical edges by comparing the left and right columns. The only differences are the `borderType` or `strides` arguments.

### Four problems in this section.

- `cv2.filter2D` **has no `strides` argument**. The stride example raises an error (I ran it). Do a normal filter and then downsample: `result[:,::2, ::2]`.
- `filter2D` **always returns an output the same size as the input**, so it cannot do true "valid" convolution. Crop `k//2` pixels from each side, or use `scipy.signal.correlate2d(image, kernel, mode='valid')`.
- The manual labels `BORDER_CONSTANT` as "valid" and `BORDER_REFLECT` as "same (zero padding)". That is backwards or at least mixed up: `BORDER_CONSTANT` pads with zeros (same convolution); `BORDER_REFLECT` mirrors the edge pixels.
- `filter2D` technically computes correlation (the kernel is not flipped). For symmetric kernels such as blur it makes no difference.

## 5. Edge detection

Manual pp. 18-20

An **edge** is a boundary where color, intensity or texture changes sharply. Detecting edges cuts the data down to the essential structure, which makes it a standard preprocessing step for detection, segmentation and feature extraction.

TECHNIQUES LISTED IN THE MANUAL

| TECHNIQUE                                      | HOW IT WORKS                                                                                                                  |
|------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <b>Gradient-based</b> (Sobel, Prewitt, Scharr) | Compute the rate of change of intensity. Edges are where gradient magnitude is high. Simple, 3x3 kernels, x and y directions. |
| <b>Canny</b>                                   | Multi-stage: Gaussian smoothing, gradient, non-maximum suppression, hysteresis. Accurate and noise-resistant.                 |
| <b>Laplacian of Gaussian (LoG)</b>             | Gaussian blur then second derivative. Edges are the zero-crossings.                                                           |
| <b>Marr-Hildreth</b>                           | LoG applied after Gaussian smoothing, locating edges at zero-crossings.                                                       |
| <b>Sobel-Feldman</b>                           | Sobel variant that also emphasizes diagonal edges.                                                                            |
| <b>CNN-based</b>                               | Learns complex edge patterns from data and adapts to a domain.                                                                |

SOBEL EDGE DETECTION, LINE BY LINE

```
gradient_x = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3)    # change left-right: finds vertical edges
gradient_y = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3)    # change up-down: finds horizontal edges

gradient_magnitude = np.sqrt(gradient_x**2 + gradient_y**2)  # combined edge strength
gradient_direction = np.arctan2(gradient_y, gradient_x)      # angle in radians

threshold = 100
edges = (gradient_magnitude > threshold).astype(np.uint8) * 255
# True/False mask to 0/1 to 0/255: white where the edge is strong, black elsewhere

cv2.imshow('Original Image', image)                        # two windows
cv2.imshow('Detected Edges', edges)
cv2.waitKey(0)                                             # wait until any key is pressed
cv2.destroyAllWindows()                                   # close windows
```



The simple threshold of 100 is a single cut-off: edges are thick and noise-sensitive. Canny fixes that.

5.2 The Canny edge detector

Manual pp. 20-22 · John F. Canny, 1986

**1. Gaussian smoothing**

Blur with a Gaussian kernel so small intensity variations (noise) do not create false edges.

**2. Gradient calculation**

Two 3x3 Sobel kernels give the x and y gradients, then magnitude and direction per pixel.

### 3. Non-maximum suppression

For each pixel, keep it only if its magnitude is the largest among neighbors along the gradient direction. This thins edges to one pixel wide.

### 4. Hysteresis tracking

Two thresholds. Above **T\_high**: strong edge, kept. Between **T\_low** and **T\_high**: weak, kept only if connected to a strong edge. Below **T\_low**: rejected.

#### PARAMETERS

- **Gaussian kernel size / sigma**: larger gives a smoother image but loses fine detail.
- **T\_high and T\_low**: trade-off between sensitivity and noise suppression. Usually found by experiment. A common ratio is about 1:2 or 1:3.

```
edges = cv2.Canny(image, threshold1=100, threshold2=200)
# threshold1 = T_low (100), threshold2 = T_high (200).
# Canny already does the Sobel gradient, non-max suppression and hysteresis internally.
# It does NOT blur for you, so call cv2.GaussianBlur first on noisy images.
# Output: binary image, 255 on edges, 0 elsewhere.
cv2.imshow('Canny Edges', edges); cv2.waitKey(0); cv2.destroyAllWindows()
```

**Effect of thresholds.** Lower both: more edges and more noise. Raise both: cleaner but may miss faint edges.

## Lab task: texture analysis for material classification

Manual p. 14. The manual asks for code and a description.

**Problem:** classify wood, metal and fabric from images using texture features. Describe at least two techniques, explain how each works and which textures suit it, and discuss advantages and limitations.

#### SUGGESTED ANSWER: USE LBP PLUS TEXTURE ENERGY/CONTRAST

```
import cv2, numpy as np
from skimage import feature

def texture_features(path):
    gray = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    gray = cv2.resize(gray, (256, 256))
    f = gray.astype(np.float32)

    # Technique 1: LBP histogram (10 values)
    lbp = feature.local_binary_pattern(gray, 8, 1, method='uniform')
    lbp_hist, _ = np.histogram(lbp, bins=np.arange(0, 11))
    lbp_hist = lbp_hist / (lbp_hist.sum() + 1e-6)

    # Technique 2: local energy and contrast in 3x3 windows
```

07/10/2026, 17:38Lab 04 Feature Extraction Guide

```
mean_sq = cv2.blur(f * f, (3, 3)) # mean of squares = energy / 9
mean = cv2.blur(f, (3, 3))
contrast = np.sqrt(np.maximum(mean_sq - mean**2, 0)) # local std deviation
e_hist, _ = np.histogram(mean_sq, bins=16, range=(0, 255**2))
c_hist, _ = np.histogram(contrast, bins=16, range=(0, 128))
e_hist = e_hist / e_hist.sum(); c_hist = c_hist / c_hist.sum()

return np.concatenate([lbp_hist, e_hist, c_hist]) # 42-value vector

# Train: X = [texture_features(p) for p in image_paths]; y = labels
# from sklearn.svm import SVC; SVC().fit(X, y)
```

### LBP: how it works and where it fits

Compares each pixel to its 8 neighbors, builds a code, histograms the codes. Good for fine repeating micro-texture: **wood grain** (directional lines), **fabric weave** (regular pattern), brushed or rough **metal**.

### Energy and contrast: how it works and where it fits

Window energy and standard deviation, then histograms. Separates **smooth, uniform** surfaces (polished metal: low contrast, narrow histogram) from **rough or busy** ones (fabric, wood: wide contrast histogram).

### Advantages

- Cheap to compute, small vectors.
- LBP is unaffected by uniform brightness changes (it only compares).
- Energy/contrast needs no training data and is easy to interpret.
- Complementary: LBP captures pattern, energy/contrast captures strength.

### Limitations

- LBP at radius 1 sees only fine detail and misses large patterns. Try several radii.
- Not rotation- or scale-invariant in the basic form, so photo distance matters.
- Energy/contrast is sensitive to lighting and exposure, and the histograms ignore spatial layout.
- Shiny metal reflections can look like texture. Fixed window size (3x3) is a compromise.

## Manual code problems at a glance

These were checked by running the code. Know them if you are asked to run the manual's code or explain odd results.

| WHERE         | PROBLEM                                                             | FIX         |
|---------------|---------------------------------------------------------------------|-------------|
| HED (p. 9-10) | Angles are -180..180 but range is (0, 360): negative angles dropped | angle % 360 |

|                             |                                                                       |                                                                                            |
|-----------------------------|-----------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| HIG (p. 11-12)              | Same: range (0, 180) drops negatives.<br>No magnitude weighting.      | <code>angle % 180</code> , <code>weights=magnitude</code>                                  |
| Texture energy (p. 13)      | <code>image ** 2</code> overflows uint8, output clipped               | Convert to float32 first                                                                   |
| Texture contrast (p. 13-14) | Computes a sum, not a standard deviation                              | $\sqrt{(\text{mean of squares} - \text{square of mean})}$                                  |
| Gaussian blur (p. 16)       | <code>getGaussianKernel</code> returns a 5x1 column                   | <code>cv2.GaussianBlur</code> or <code>k @ k.T</code>                                      |
| Valid conv (p. 17)          | <code>filter2D</code> returns the full size, so it is not valid conv  | Crop borders or <code>correlate2d(mode='valid')</code>                                     |
| Same conv (p. 17-18)        | Labelled zero padding, but uses <code>BORDER_REFLECT</code>           | <code>BORDER_CONSTANT</code> is zero padding                                               |
| Strides (p. 18)             | <code>filter2D</code> has no <code>strides</code> : raises an error   | <code>result[::2, ::2]</code>                                                              |
| Objectives (p. 3)           | Objective 6 is corner detection, but the manual has no corner section | Harris or Shi-Tomasi: <code>cv2.cornerHarris</code> , <code>cv2.goodFeaturesToTrack</code> |

Based on the Lab 04 Manual (AI-4002). Code comments and the notes on bugs are additions for study.

